

מבדק פיתוח Stack Full - מפתח/ת תוכנה רמה ג'

משימת פיתוח להערכת יכולת מקצועית, איכות קוד, חשיבה מערכתית, תכנון עבודה ויכולת קבלת החלטות טכנולוגיות.

המטרה אינה לבנות מערכת גדולה או להשקיע בעיצוב גרפי, אלא להציג פתרון מקצועי, ברור, יעיל וניתן לתחזוקה, ברמה המתאימה לסביבת פיתוח ארגונית.

מטרת המבדק

- .NET / Core .NET - תכנון שכבות, Injection Dependency, עבודה אסינכרונית וטיפול בשגיאות
- API REST - תכנון חוזה API, סטטוסים, Validation, Pagination, Filtering ו-Sorting
- Angular - ארכיטקטורת רכיבים, RxJS ועבודה נכונה מול API
- בסיס נתונים - מודל, אינדקסים, שאילתות יעילות ו-Aggregations
- Concurrency, ביצועים ואמינות
- בדיקות אוטומטיות משמעותיות
- תכנון עבודה וחלוקה נכונה של האפיון למשימות פיתוח
- יכולת לנמק החלטות טכנולוגיות וחלופות

המשימה

יש לפתח מערכת לניהול פניות/בקשות המתקבלות מארגונים ומעסיקים. המערכת מיועדת לעבודה של מספר משתמשים במקביל ותכיל כמות גדולה של פניות. המשתמשים יוכלו לאתר פניות, לסנן, לצפות, לעדכן סטטוס ולצפות בנתונים מסכמים.

מבנה פנייה

- Id - מזהה ייחודי
 - Title - כותרת הפנייה
 - OrganizationName - שם הארגון/העסק
 - Status - New, InProgress, Waiting, Completed
 - Priority - Low, Medium, High
 - AssignedTo - מזהה/שם מטפל
 - CreatedAt - תאריך יצירה
 - UpdatedAt - תאריך עדכון אחרון
 - RowVersion / Version - שדה המאפשר זיהוי עדכון מתחרה
- יש ליצור לפחות 100,000 רשומות לצורך בדיקת המערכת. בסיס הנתונים לבחירת המועמד/ת: Server SQL, MongoDB או חלופה אחרת. יש לאפשר יצירה חוזרת של נתוני הבדיקה.

1. שליפת נתונים - חובה

יש לפתח API REST באמצעות .NET / Core .NET לשליפת פניות. אין להעביר את כלל הרשומות לClient- לצורך סינון, מיון או דפדוף.

- Pagination בצד השרת.

- סינון משולב לפי לפחות Status, Priority, OrganizationName, AssignedTo וטווח תאריכים.

- חיפוש טקסטואלי לפחות בTitle- ובOrganizationName-.

- מיון לפי לפחות שלושה שדות, כולל Asc/Desc.

- לפחות שני Aggregations רלוונטיים.

- הגבלת Size Page וValidation- לפרמטרים.

- CancellationToken בפעולות I/O רלוונטיות.

- החזרת DTOs ולא ישויות DB ישירות.

יש לתעד כיצד נמנעת טעינת כלל הנתונים לזיכרון וכיצד האינדקסים תומכים בתרחישי החיפוש והסינון המרכזיים.

2. עדכון סטטוס ותחרותיות - חובה

- פנייה שאינה קיימת - קוד HTTP מתאים.

- סטטוס שאינו חוקי או מעבר סטטוס שאינו מותר - Validation ברור.

- Concurrency Optimistic: בעדכונים מתחרים רק עדכון תקין אחד יצליח והאחר יקבל Conflict מתאים.

- UpdatedAt וVersion/RowVersion- יעודכנו באופן עקבי.

- יש להימנע Update-Lost.

3. היסטוריית שינויים - חובה

לכל שינוי Status יש לשמור Audit בסיסי הכולל לפחות RequestId, PreviousStatus, NewStatus, ChangedAt וChangedBy- . יש לספק Endpoint לצפייה בהיסטוריית השינויים של פנייה.

4. פעולה Bulk - חובה

יש לממש Endpoint לעדכון סטטוס של מספר פניות באותה בקשה, עד 100 פניות. יש להגדיר ולתעד את ההתנהגות כאשר חלק מהפניות אינן קיימות או נכשלות עקב Concurrency.

ניתן לבחור בין All-or-Nothing לבין Success Partial, אך יש לנמק את הבחירה ולהחזיר תשובה ברורה ללקוח.

5. Angular - חובה

- חיפוש, סינון, מיון ודפדוף בצד השרת.
- Debounce לחיפוש.
- ביטול/החלפת בקשות קודמות כאשר החיפוש משתנה במהירות.
- Loading, State Empty, State -Error.
- עדכון סטטוס וטיפול ב-409 Conflict.
- הצגת Aggregations.
- צפייה בהיסטוריית שינויים.
- הפרדת אחריות בין Components ו-Services.

6. ביצועים - חובה

- יש לבחור לפחות שתי פעולות API מרכזיות ולתעד כיצד נבחנה יעילותן על בסיס 000,100 הרשומות.
- Explain, Plan Query/Execution או מדד מקביל אחד לפחות.
- פירוט האינדקסים שנוספו והסיבה.
- זיהוי Bottleneck אפשרי והצעת שיפור.
- אין לבצע טעינה מלאה לפני Filtering/Pagination כאשר ניתן לבצע את הפעולה בבסיס הנתונים.

7. Cache - חובה ברמה מוגבלת

- יש לממש Cache עבור נתון מסכם אחד לפחות או תרחיש קריאה אחר שבו קיימת הצדקה.
- הגדרת Expiration.
- מניעת מידע מיושן לאחר עדכון באמצעות Invalidation מתאים.
- נימוק לבחירת המידע שנשמר ב-Cache.
- תיאור התאמה נדרשת בסביבה עם מספר מופעי שרת.

8. בדיקות אוטומטיות - חובה

- יש לצרף לפחות 4 בדיקות אוטומטיות משמעותיות. לפחות אחת מהן Test Integration.
- תרחיש תקין של שליפה/סינון או עדכון.
- Validation / קלט לא תקין.
- Conflict Concurrency.
- Test Integration הכולל API ובסיס נתונים או תחליף מתאים.

9. תכנון עבודה וחלוקה למשימות - חובה

כחלק מהמבדק יש להציג כיצד האפיון תורגם לתוכנית עבודה ולמשימות פיתוח. אופן הצגת התכנון נתון לבחירת המועמד/ת.

- חלוקה הגיונית של הדרישות ליחידות עבודה ברורות, לדוגמה Task / Story User / Feature או מבנה מקביל.
- לכל משימה יש להגדיר בקצרה מה נדרש לביצוע ומה התוצר המצופה.
- יש להציג סדר ביצוע ותלויות בין משימות, ככל שקיימות.
- יש לציין הערכת מאמץ לכל משימה או יחידת עבודה.
- יש להראות קשר ברור בין דרישות המבדק לבין המשימות שנפתחו.
- יש לכלול גם משימות שאינן רק כתיבת קוד, כגון תכנון בסיס נתונים, בדיקות, ביצועים, תיעוד והיערכות להרצה.

המטרה היא לבחון יכולת לפרק אפיון לפיתוח בצורה מסודרת ומעשית, ולא רק להציג מוצר סופי.

10. איכות הקוד

- מבנה פרויקט ברור והפרדת אחריות.
- קוד קריא, Naming עקבי והימנעות משכפול.
- Mapping / Models / DTOs באופן ברור.
- Injection Dependency ושימוש נכון ב-Lifetimes.
- עבודה אסינכרונית נכונה.
- Details Problem / Handling Error Global או פתרון מקביל.
- Validation עקבי.
- Logging בנקודות משמעותיות ללא חשיפת מידע רגיש.
- הימנעות מ-Over Engineering.

11. תיעוד הפתרון - חובה

- הוראות ברורות המאפשרות להבין ולהפעיל את הפתרון.
- טכנולוגיות וגרסאות.
- תיאור מבנה הפתרון.
- בחירת בסיס הנתונים והאינדקסים.
- הסבר על Concurrency ועל Update Bulk.
- הסבר על Cache ו-Invalidation.
- שתי החלטות טכנולוגיות משמעותיות וחלופות שנשקלו.
- מגבלה ידועה אחת ודבר אחד שהיה ניתן לשפר.
- פירוט שימוש בכלי AI - באילו חלקים נעשה שימוש ומה נבדק או שונה ידנית.

שימוש בכלי AI

השימוש בכלי AI במהלך ביצוע המבדק מותר ואינו מוריד ניקוד או מהווה יתרון בפני עצמו. המועמד/ת אחראי/ת לכל קוד המוגש במסגרת המבדק.

אופן ההגשה

אופן הצגת והנגשת הפתרון לבודקים נתון לבחירת המועמד/ת. יש לוודא כי הפתרון מאפשר לבודקים לבחון באופן מלא ונוח את המערכת, קוד המקור, תכנון העבודה, אופן המימוש והחלטות הפיתוח.

ההגשה צריכה לכלול את כלל המרכיבים הנדרשים במבדק ולאפשר בחינה והרצה של הפתרון.

לא תתקבל שליחת קובצי ZIP או קוד מקור כקובץ מצורף בדוא"ל.

אופן ההגשה, הנגישות לפתרון, שלמותו ואיכות הצגתו יהוו חלק מהערכת המבדק.

אופן הערכת המבדק

מה נבדק	משקל	תחום הערכה
מבנה, DI, Async, טיפול בשגיאות, לוגיקה ו-Logging	17	Core .NET / .NET – Backend
HTTP, Filtering, Pagination, Validation ו-Bulk	13	API REST
Components, Services, RxJS, Server-side operations ומצבי UI	15	Angular
מודל, שאילתות, אינדקסים, Aggregations ומידת ביצועים	13	בסיס נתונים וביצועים
Concurrency Optimistic, Conflict, עקביות והיסטוריה	11	Audit + Concurrency
פירוק אפיון, Tasks, תלויות, סדר ביצוע והערכת מאמץ	8	תכנון עבודה וחלוקה למשימות
תחזוקתיות, הפרדת אחריות והחלטות טכנולוגיות	9	איכות קוד וחשיבה מערכתית
בחירה, Expiration, Invalidation ו-Scale-out	4	Cache
תרחישים משמעותיים כולל Integration ו-Concurrency	6	בדיקות אוטומטיות
בהירות, נגישות, החלטות, AI ושלמות ההגשה	4	תיעוד ואיכות ההגשה
	100	סה"כ

דגשים להערכה

- פתרון שעובד אינו בהכרח פתרון שיקבל את מלוא הניקוד.
- מימוש המבצע סינון, מיון או דפדוף של כלל הנתונים ב-Client – ייפגע משמעותית בציון.
- Concurrency המטופל רק ברמת UI ואינו נאכף בצד השרת או בסיס הנתונים אינו פתרון מלא.
- Cache ללא אסטרטגיית Invalidation לא יקבל את מלוא הניקוד.
- חלוקת משימות שטחית שאינה משקפת את האפיון, התלויות והעבודה הנדרשת לא תקבל את מלוא הניקוד.
- שימוש ב-AI אינו מחליף איכות קוד, נכונות, תכנון עבודה וחשיבה מערכתית.
- Engineering Over אינו יתרון; פתרון פשוט, נכון ומנומק עדיף על מורכבות ללא צורך.

בהצלחה!